

Hardware-Aware TinyML for Energy-Efficient Cataract Detection in Ophthalmology on Edge Devices

Abhishek Kakde

Department of Computer Science and Engineering

NIIT University

Neemrana, India

abhishek.kakde23@st.niituniversity.in

Naman Mani

Department of Computer Science and Engineering

NIIT University

Neemrana, India

naman.mani23@st.niituniversity.in

Riya Gupta

Department of Computer Science and Engineering

NIIT University

Neemrana, India

riya.gupta23@st.niituniversity.in

Vikas Upadhyaya

Electronics and Communications Engineering

NIIT University

Neemrana, India

vikas.upadhyaya@niituniversity.in

Abstract—The growth of edge devices demands intelligent inference at the point of acquisition, yet most existing or currently designed models are built with GPU or AI accelerator based infrastructure and the consumption of memory and power are far excessive than the constraints of microcontroller class devices. Existing TinyML research is generally divided into two broad classes. The first class largely focuses on evaluation of model accuracy in isolation with limited focus on deployment metrics that govern real world feasibility, including inference latency, flash memory footprint, and energy per inference on edge devices. The other class tends to work on simple or testing projects with less concern for models that can solve real world problems. This paper presents a hardware-aware TinyML pipeline for low-power deep learning model deployment on edge devices. The pipeline uses MobileNetV2 fine tuning, post training INT8 quantization, conversion to TensorFlow Lite, and benchmarking of deployment and model evaluation parameters through a binary cataract classification task as a representative medical screening application. The dataset consisted of 5678 image samples. After applying data augmentation techniques, the number of effective training samples increased to 11,922 samples. The quantized MobileNetV2 model showed a reduced inference latency of 29.76% and a reduction of 71.79% in model size, while there was a drop in accuracy of just 2.48% compared to the Float Keras model. Further, the INT8 TensorFlow Lite model was deployed on Raspberry Pi ARMv7 hardware, where an average latency of 451.61 ms and an accuracy of 93.19% was observed. The average power consumption of a single inference was noted to be 5.11 Watts. Although latency increased in a real edge device, the results showed that the deployment of quantized deep learning models for medical applications on edge devices can be feasible.

Index Terms—TinyML, edge AI, MobileNetV2, TensorFlow Lite, INT8 quantization, cataract classification, low-power deployment

I. INTRODUCTION

The growth of Artificial Intelligence in today's era has created new benchmarks in the medical diagnosis field. AI-

assisted diagnostics are becoming a major change in today's digital era. These treatment modes, however, are operating on cloud connectivity, GPU hardware, and excessive power supply, creating a demand to reduce power usage on these devices so that AI-assisted treatment can be deployed effectively and efficiently in limited resource environments.

Conditions such as cataract and diabetic retinopathy are leading diseases which are preventable when discovered at early stages, but they still remain leading causes of blindness globally. State of the art deep learning models for image classification in the ophthalmology domain promise high accuracy, but the cost includes heavy electricity bills and large memory and compute requirements. Models such as the Temporal Aware Hybrid Deep Learning (TAHDL) framework occupy approximately 170 MB and require GPU execution. They use ensemble architectures which combine VGG-16 and InceptionV3 and increase the power requirements further. On the other hand, microcontroller class devices typically occupy 256 KB to 4 MB of flash memory and operate at milliwatt level power consumption, which makes them an ideal target for efficient deployment of machine learning models.

TinyML [4] is a machine learning technology and application space including algorithms, hardware, and software capable of performing on-device operation at extremely low power consumption, typically in the milliwatt range and below, enabling a variety of always-on machine learning use cases on battery operated devices. However, existing TinyML medical imaging works barely report proper deployment metrics for the model used. Kumar et al. [7] achieved 82.37% accuracy using a ResNet-101 model mentioned as a TinyML deployment, but reported no figures for latency and power. Further, Asare et al. [8] deployed an INT8 quantized MobileNet via Edge Impulse, but did not analyze energy consumption, and their reported

training accuracy of 96.45% with 73.58% validation accuracy raises questions regarding generalization. To our knowledge, no prior ophthalmic TinyML study reports deployment metrics such as energy per inference for all model formats including INT8 and Flex TFLite on the same dataset.

This paper presents a hardware-aware TinyML pipeline for low power deep learning on edge devices. Rather than proposing a new application, this work focuses more on evaluating post-training INT8 quantization and its effects on accuracy, latency, model size, and energy consumption. MobileNetV2, with its depthwise convolution structure and native compatibility with TensorFlow Lite, was selected as the base architecture. The task chosen for validation is binary cataract classification.

Key Contributions:

- A complete TinyML pipeline covering dataset preparation, data analysis, MobileNetV2 [1] fine tuning, INT8 quantization, conversion to TensorFlow Lite, and hardware benchmarking.
- A comparison of model evaluation parameters and deployment metrics for two models, namely Float Keras and INT8 TFLite, across the same dataset.
- Real hardware deployment on Raspberry Pi ARMv7 microcontroller, measuring an accuracy of 93.19%, and 2.31 Joules energy per inference.

II. RELATED WORK

A. Deep Learning for Ophthalmic Disease Screening

A significant amount of research demonstrates automated eye disease diagnosis using Convolutional Neural Networks, yet most studies focus on increasing the predicted accuracy while leaving deployment feasibility aside. Sushith et al. [14] proposed the use of a Temporal Aware Hybrid Deep Learning model which combines CNN based spatial feature extraction with LSTM temporal modelling and an attention mechanism to capture disease development. The model was evaluated against three datasets, DRIVE (97.5% accuracy), EyePACS (96.9% accuracy), and Kaggle DR (94.04% accuracy), showing strong recognition, but the complete model weighs 170 MB with 10–12 GB GPU memory requirement, which makes it unfeasible for embedded deployment. A quantized or lightweight variant of the model was not proposed.

Suryawanshi and Reddy applied YOLOv9 on an 888 image dataset and reported confidence scores above 0.84 for real-time cataract detection. While they proposed a framework for telemedicine deployment, the work did not report complete accuracy figures, latency measurements, or energy consumption analysis. Amutha et al. [3] proposed a hybrid cataract classifier for resource constrained rural healthcare using a custom 6-layer CNN with 2D Fourier transform, reporting an accuracy of 90% on four severity levels on 400 images, but did not profile inference time or energy and power consumption. Ashok et al. combined VGG16 with InceptionV3 through transfer learning, reporting an approximate accuracy of 98.1% on OCT classification, but the cost of running two heavy

architectures in parallel is incompatible with MCU class flash memory constraints.

B. Lightweight Architectures and Mobile Edge Deployment

A second group of studies explored lightweight CNNs on mobile and edge platforms but were unable to provide MCU level validation. Kumar et al. [7] evaluated six ResNet variants and achieved an accuracy of 82.37% on the APTOS 2019 dataset after preprocessing steps such as CLAHE and knowledge distillation for compression. However, the paper did not report inference latency and SRAM usage, which are precisely the metrics required for viable MCU class deployment. Asare et al. [8] focused on multi-model deployment and reported accuracy for several architectures, including MobileNet (3.4 MB, 73.58% validation accuracy, 19 ms GPU latency), ShuffleNet (68.5 KB, 68.47%), SqueezeNet (176 KB, 72.59%), and a custom DNN via Edge Impulse with INT8 quantization. The study was deployment oriented, but it had two limitations. First, energy consumption was not reported. Second, the large gap between training accuracy of 96.45% and validation accuracy of 73.58% indicates overfitting. Kavitha and Sridevi [9] proposed Residual Attention U-Net achieving a dice score of 0.87 and exported the model to TensorFlow Lite for latency below 200 ms. However, quantization was not performed, leaving a gap between mobile export and embedded deployment.

C. TinyML Systems and Hardware-Aware Pipelines

Fasolino et al. [10] deployed a two-layer CNN on STM32 microcontrollers for classification of four object groups using an additional 8×8 time-of-flight depth map. They achieved 92.61% accuracy with a model footprint of 2.65 KB, sub-millisecond inference, and energy consumption of 32 μ J per inference. The deployment is directly relevant to MCU class deployment, but the domain and model were for simple object detection rather than a complex medical imaging problem. Similarly, Zhuang and Pau [11] proposed a tiny DNN framework for MCU deployment using Bayesian optimization for joint architecture search and post-training quantization. They achieved a $4\times$ memory reduction from 4.17 MB to 0.716 MB on CIFAR-10, but the model did not report energy metrics, nor was it evaluated on a medical imaging task.

III. PROPOSED SOLUTION AND METHODOLOGY

A. Proposed Solution

This section presents the proposed system and the ideas which shaped the design of the proposal “Low power hardware-aware TinyML pipeline.” The system is designed around the impact it makes on hardware metrics while keeping a balanced classification accuracy at the same time.

1) *Design Philosophy:* The proposed system is intended to be deployment first work. Unlike prior work on ophthalmic AI, where deployment metrics, practical application, and resource constrained environments are often missing, the system is designed to optimize the tradeoff among accuracy, latency, and energy, together with deployment and hardware-aware metrics for final deployment on microcontroller class devices.

2) *System Architecture*: The proposed system is a hybrid mobile-edge inference architecture where image acquisition is decoupled from model inference. Images are captured and preprocessed on the mobile interface, including resizing to 224×224 pixels and pixel value normalization. The pre-processed image tensor is then forwarded to the embedded inference engine. The TensorFlow Lite model resides on the inference engine and, on receiving the image tensor, returns the predicted class, which is finally displayed on the mobile device. A Universal Serial Bus cable is used as the medium between the mobile device and inference engine, which transfers the tensor and predicted class.

The embedded inference engine uses a microcontroller unit belonging to the Raspberry Pi family. The INT8 TFLite model weighs 2.58 MB and is compatible with Raspberry Pi series devices supplemented with external flash.

3) *Quantization Strategy*: The system demonstrates post-training quantization for two reasons. First, post-training quantization does not require access to the full training pipeline after deployment. Second, post-training quantization on MobileNetV2 has shown degradation in accuracy of well below 1% for image classification tasks. The proposed pipeline converts the trained Keras model into TensorFlow Lite INT8, using the TensorFlow Lite converter. In the INT8 model, all weights and activations are quantized to 8-bit integers using a representative calibration subset. The INT8 model is benchmarked against the Float Keras baseline, providing a comparison across the accuracy-efficiency frontier.

B. Methodology

1) *Dataset selection and Pre-processing*: The dataset was combined from three Kaggle repositories. Eye Diseases Classification dataset (2,112 images), the Cataract Dataset (400 images), and the ODIR-5K dataset (3,166 images), obtaining 5,678 unique fundus images. Low quality images were preserved to test the robustness of the model. All images were resized to 224×224 pixels so that they match the input requirement of MobileNetV2. The pixel values were normalized between the range [0,1].

Data augmentation techniques like horizontal flipping, brightness adjustment, and random rotation were applied during training so that the model does not overfit and has better generalization. The total number of training samples were 11,922 augmented images. The final test was done on a set of 852 samples.

2) *Model Selection and Training*: Keeping in mind the depthwise separable convolutions and inverted residual structure of MobileNetV2, it was chosen as the base model as it provided a good tradeoff between accuracy and parameters among all the lightweight CNNs. MobileNetV2 is also compatible with TensorFlow Lite INT8 quantization which is an important step in reducing the model size. The model was fine tuned using **Adam optimizer** and the **categorical cross-entropy loss**

3) *Optimization Pipeline*: The model was saved in Keras format after training was done and converted into variants of optimized TensorFlow Lite via the TensorFlow Lite converter.

- **INT8 Quantized TensorFlow Lite Model**: Both weights and activations were converted to 8-bit integers in which full integer post-training quantization was applied. Scaling factors were decided using a calibration dataset which ensured minimal accuracy degradation.

4) *Evaluation Metrics*: Both the models obtained after the optimization step are evaluated across two dimensions. Classification performance is measured using accuracy, precision, recall, and F1-score on the 852 sample test set. Hardware efficiency is measured using inference latency in milliseconds, model size in KB/MB. Average Power Consumption is estimated as

$$P = 2.8 + \frac{C}{100} \times 3.2 \quad (1)$$

where C is average CPU usage, 2.8 W is the idle power draw of the device, and 3.2 W is the additional power required for full CPU load.

And Energy per inference is estimated as

$$E = P \times t \quad (2)$$

where Power is calculated from Equation 1 and t is inference time in seconds.

IV. RESULTS AND DISCUSSION

The proposed TinyML pipeline is evaluated across both classification performance and deployment efficiency. The models; Float Keras and INT8 quantized TensorFlow Lite are compared with each other. The objective is to analyze the trade-off between hardware-aware metrics like model size, energy consumption and latency, and the predictive accuracy under microcontroller constraints.

A. Classification Performance

Table I presents the classification metrics and hardware measurements for both the model variants.

TABLE I
MODEL PERFORMANCE COMPARISON

Model	Acc.	Prec.	Rec.	F1	Latency	Size
Keras	0.948	0.991	0.938	0.964	19.78 ms	9.17 MB
TFLite	0.924	0.993	0.905	0.947	13.89 ms	2.59 MB

The Float Keras model achieved a baseline accuracy of 94.84% with an inference latency of 19.78 ms and a model size of 9.17 MB. The INT8 TFLite model shows near-identical accuracy of 92.49% with reduced latency of 13.89 ms and a drastic reduction of 71.1% in model weight to 2.59 MB.

The confusion matrices in Table II show that quantization does not redistribute the errors unevenly across classes. Compared to the float baseline, the INT8 model produced 52 false negatives and 47 false positives on the 852 sample test set,

while the baseline model gave 39 false negatives and 5 false positives.

The INT8 model was deployed on Raspberry Pi ARMv7 microcontroller. Table III reports the results and the latency, energy and power metrics obtained from the microcontroller.

TABLE II
COMPARISON OF CONFUSION MATRICES FOR DIFFERENT MODELS

	Float Keras		INT8 TFLite	
	Pred. 0	Pred. 1	Pred. 0	Pred. 1
Actual 0	210	5	211	4
Actual 1	39	598	52	585

TABLE III
CLASSIFICATION RESULTS ON RASPBERRY PI ARMV7 (852 TEST IMAGES)

Metric	Value
Accuracy	93.19%
Precision	94.35%
Recall	93.19%
F1-Score	93.41%
Average Inference Latency	451.62 ms
Average CPU Utilisation	72.33%
Average Power Consumption (Eq. 1)	5.114 W
Average Energy per Inference (Eq. 2)	2.31 J

B. Comparison to Prior Work

Table IV positions the proposed system against the most relevant related work reported in [3], [7], [8], and [10].

TABLE IV
COMPARISON WITH PRIOR WORK

Study	Architecture	Accuracy	Energy	MCU
Asare et al.	MobileNet	73.6%	No	Partial
Kumar et al.	ResNet-101	82.4%	No	No
Fasolino et al.	Tiny CNN	92.6%	Yes	Yes
Amutha et al.	CNN+2D-DFT	90.0%	No	No
Proposed	MobileNetV2	93.19%	Yes	Yes

The proposed system reports an accuracy of 93.19%, substantially higher than both Asare et al. [8] and Fasolino et al. [10] in comparable edge oriented studies, while also being one of the few ophthalmic TinyML works to report energy per inference.

V. CONCLUSION

The paper looks over the problem of deep learning inference on low power devices which are Micro-controller class devices, the primary challenge being the deployment feasibility and awareness whilst secondary goals involves model accuracy. Through a hardware aware TinyML pipeline built on MobileNetV2, post training INT8 quantization and deployment in TFLite format paper demonstrates that the rigidity of efficiency of embedded model and predictive performance is resolvable without retraining and architectural design. Of the two models Float Keras & INT8 quantized

model, INT8 model emerged as most optimal deployment candidate across all dimension: accuracy, latency, model size and energy efficiency. More importantly this benchmarks are achieved by deploying the model on a Raspberry Pi ARMv7 microcontroller. The numerical results include a drop of 2.48% accuracy of INT8 TFLite model compared to float Keras model and energy cost of 2.31 Joules per inference. On parameters as inference latency there was a reduction of 29.76% and drop of Model size by 71.79%. The paper establishes a reproducible benchmarking framework which evaluates classification accuracy, inference latency, model size and energy consumption together, a combination absent from prior ophthalmic TinyML literature.

A. Limitations

There are a few limitations despite the work done. The dataset, even though modified to 5,678 images, is limited to binary classification, real world application requires multi-class classification. Thermal throttling was observed during Raspberry Pi inference, this indicates that deployment for a longer time might require active cooling system and thermal management. Future work should deploy the model on other class microcontrollers such as STM32 to validate model specific energy requirement.

REFERENCES

- [1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, doi: 10.1109/CVPR.2018.00474.
- [2] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Mach. Learn. (ICML), 2019, doi: 10.48550/arXiv.1905.11946.
- [3] M. Amutha, A. Deepika, and R. Divya Dharshini, "Real-time ML-based cataract detection in resource-constrained environments," in Proc. IEEE Int. Conf. Power, Control, Signals and Instrumentation Engineering (ICPCSN), 2025, doi: 10.1109/ICPCSN65854.2025.11035195.
- [4] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA: O'Reilly Media, 2019.
- [5] J. Lin, W.-M. Chen, Y. Lin, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, doi: 10.48550/arXiv.2007.10319.
- [6] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," arXiv:2010.08678, 2020, doi: 10.48550/arXiv.2010.08678.
- [7] M. Kumar, H. P. Mohan Kumar, H. Sultana, and J. S. Shweta, "Mobile-based classification and detection of diabetic retinopathy using TinyML," in Proc. IEEE Nat. Conf. Knowl., Comput. Commun. (NKCon), 2023, doi: 10.1109/NKCon59507.2023.10396193.
- [8] A. Asare, D. A. N. Gookyi, D. Boateng, and F. A. Wulnye, "Deploying and evaluating multiple deep learning models on edge devices for diabetic retinopathy detection," in Proc. IEEE Int. Conf., 2024, doi: 10.48550/arXiv.2506.14834.
- [9] N. Kavitha and S. Sridevi, "Optimized deep learning architecture for precise segmentation and early diagnosis of diabetic retinopathy," in Proc. IEEE Int. Conf. Next-Gen. Comput. Syst. (ICNGCS), 2025, doi: 10.1109/ICNGCS64900.2025.11183414.
- [10] A. Fasolino *et al.*, "Object classification using ultra-low-resolution time-of-flight sensor and tiny CNN," in Proc. IEEE Sensors Appl. Symp. (SAS), 2024, doi: 10.1109/SAS60918.2024.10636597.
- [11] B. Zhuang and D. Pau, "Framework for designing tiny DNNs on MCUs," in Proc. IEEE Int. Conf. Consum. Electron. (ICCE), 2024, doi: 10.1109/ICCE59016.2024.10444435.
- [12] G. Samaras *et al.*, "Automation of TinyML for edge computing," in Proc. IEEE SmartNets, 2024, doi: 10.1109/SmartNets61466.2024.10577687.

- [13] K. Kopparapu *et al.*, "TinyFedTL: Federated transfer learning on tiny IoT devices," in Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops (PerCom Workshops), 2022, doi: 10.1109/PerComWorkshops53856.2022.9767250.
- [14] M. Sushith, A. Sathya, V. Kalaipoonguzhali, and V. Sathya, "A hybrid deep learning framework for early detection of diabetic retinopathy," *Sci. Rep.*, vol. 15, art. no. 99309, 2025, doi: 10.1038/s41598-025-99309-w.
- [15] K. Dong, C. Zhou, Y. Ruan and Y. Li, "MobileNetV2 Model for Image Classification," 2020 2nd International Conference on Information Technology and Computer Application (ITCA), Guangzhou, China, 2020, pp. 476-480, doi: 10.1109/ITCA52113.2020.00106.
- [16] Bhushan, C.M., Koppuravuri, P., Prasanthi, N. et al. Deploying TinyML for energy-efficient object detection and communication in low-power edge AI systems. *Sci Rep* 15, 44299 (2025). <https://doi.org/10.1038/s41598-025-27818-9>.
- [17] Agbo-Ajala, J.O., Akinyemi, L.A., Ekundayo, O.S. and Mnkandla, E. (2026). TinyML for Real-Time Medical Image Classification and Diagnosis. In *Tiny Machine Learning* (eds A.L. Imoize, D.-T. Do and H. Song). <https://doi.org/10.1002/9781394294572.ch15>.
- [18] S. Kim, G. Park and Y. Yi, "Performance Evaluation of INT8 Quantized Inference on Mobile GPUs," in *IEEE Access*, vol. 9, pp. 164245-164255, 2021, doi: 10.1109/ACCESS.2021.3133100.
- [19] E. Li, L. Zeng, Z. Zhou and X. Chen, "Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing," in *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447-457, Jan. 2020, doi: 10.1109/TWC.2019.2946140.